NI1 · PROGRAMAÇÃO ORIENTADA A OBJETOS

Space Invaders

Um jogo em JavaScript com canvas: da primeira versão gerada por IA à versão lateral com 5 hordas de invasores.





ANDRÉ SEIJI SHIROMA

MARCOS ALMEIDA DALOIA

CLAYTON SANTOS FERREIRA

ENZO MANCIO

APRESENTA · ENZO

01 · ESCOLHA DA IA

Duas IAs, o mesmo pedido

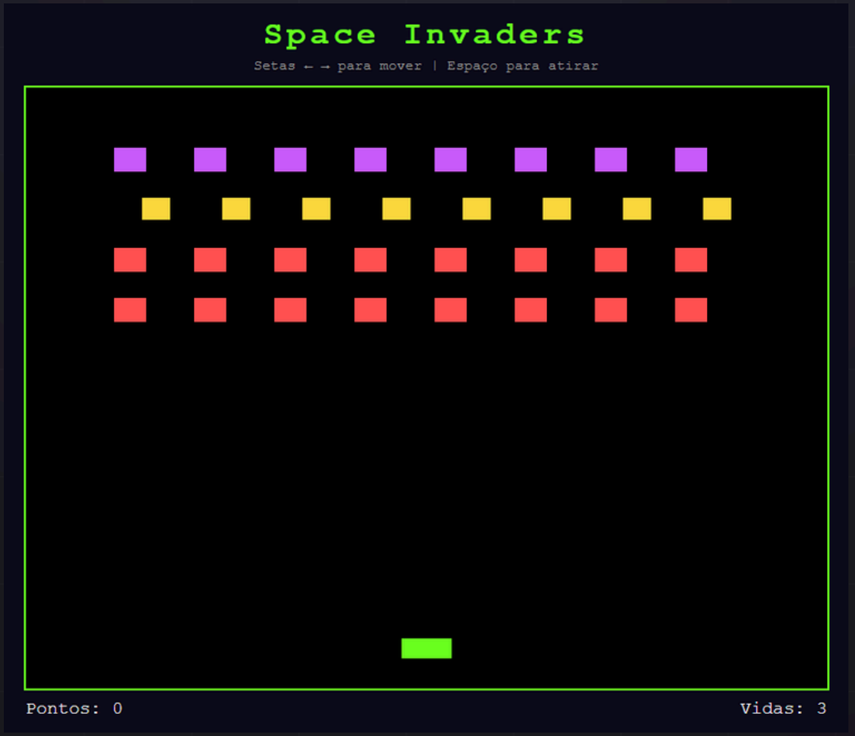
Na primeira versão do jogo, usamos o Claude e o Gemini para gerar um Space Invaders que atendesse ao enunciado da NI1.

	Claude	Gemini
ENTRADA	Enunciado da NI1 em PDF	Print + "jogo estilo space invaders"
PROMPTS	3: 2 para o tema, 1 para o jogo	1: o tema já veio no pedido
ARQUIVOS	10: HTML, CSS e 8 classes	3: tudo em um script.js
LINHAS DE JAVASCRIPT	302	166
HERANÇA	2 níveis, 3 tipos de inimigo	1 nível, 1 tipo de inimigo
IDIOMA	Português do Brasil	Português de Portugal ("ecrã")

APRESENTA · ENZO

A primeira versão de cada uma

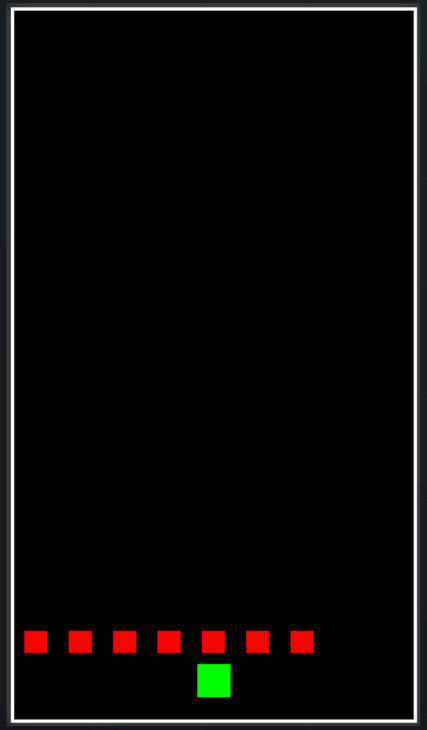
CLAUDE



9 de 11

funcionalidades: vidas, pontos, inimigos que atiram, vitória e derrota.

GEMINI



Os inimigos chegam na nave, passam por ela e continuam descendo. Ao destruir todos, a tela fica só com a nave.

3 de 11

Os inimigos não atiram. Não há vidas, pontos, vitória nem derrota.

01 · ESCOLHA DA IA

Por que seguimos só com o Claude

01 Jogo completo desde o início

Vidas, pontos, inimigos que atiram, vitória e derrota. O grupo pôde melhorar o jogo em vez de construir o básico.

02 POO de verdade

Herança em 2 níveis e polimorfismo real: mover() e tentarAtirar() mudam em cada tipo de inimigo.

03 Um arquivo por classe

Cada integrante mexe na sua parte sem conflito. O Gemini colocou tudo em um único script.js.

04 Conversa em português do Brasil

O Claude ajudou a escolher o jogo e a ferramenta antes de gerar o código.

O Gemini entregou um jogo simples demais: não dá para perder, ganhar nem reiniciar.

PARA GERAR O CÓDIGO, FOI 1 PROMPT EM CADA IA. OS OUTROS 2 DO CLAUDE SERVIRAM PARA ESCOLHER O TEMA.

APRESENTA · ENZO

02 · O JOGO POR DENTRO

Como o jogo evoluiu

01
Base gerada pelo Claude
CLAUDE (IA)

02
Telas de início e de fim
ANDRÉ

03
Escudos e velocidade
MARCOS

04
Botão de pausa
CLAYTON

05
Visual em pixel art
CLAYTON

VERSÃO LATERAL

06
Jogo virado de lado
ANDRÉ

07
Setas e escudos
MARCOS

08
5 hordas e ajuste final
ANDRÉ

09
Som e música
CLAYTON

10
Melhorias no dano
Enzo

Como pedimos para a IA

#	CONTRIBUIÇÃO	QUEM	PEDIDO AO CLAUDE (RESUMO)	O QUE A IA ENTREGOU
01	**Base do jogo**	GRUPO	Ideias de jogo, confirmar o VS Code e montar o jogo	10 arquivos, 8 classes
02	**Telas de início e fim**	ANDRÉ	Tela de início e tela de reinício ao ganhar ou perder	Classe Tela e 3 estados
03	**Escudos e velocidade**	MARCOS	Melhorar o jogo e listar o que mudou	Escudos e passo fixo
04	**Pausa**	CLAYTON	Ideias de jogo, confirmar o VS Code e montar o jogo Criar um botão de pausa	Estado de pausa no loop
05	**Pixel art**	CLAYTON	Desenhar a arte	Sprites desenhados no canvas
06	**Jogo virado de lado**	ANDRÉ	Jogo na horizontal, tela cheia e 4 direções; inimigos em pé	11 arquivos alterados
07	**Setas e escudos**	MARCOS	Setas iguais ao WASD e escudos bloqueando a nave e os tiros	Teclas unificadas e colisão
08	**5 hordas e ajuste final**	ANDRÉ	5 hordas, vidas restauradas; mais atiradores; horda 5 menor	Tabela Jogo.HORDAS
09	**Som e música**	CLAYTON	Efeitos de tiro e explosão e música de fundo	Sons e trilha integrados

Arquitetura: quem herda de quem



HERDA (EXTENDS)

COMPOSIÇÃO: O JOGO TEM UM...



TEM UM (COMPOSIÇÃO)

12 classes, uma por arquivo. Os desenhos ficam no objeto Sprites, usado por todas as entidades.

Herança e polimorfismo

INIMIGO.JS · INIMIGORAPIDO.JS

```
class Inimigo extends Entidade {
  mover(velocidadeBase) {
    this.y += velocidadeBase * this.direcao;
  }
  tentarAtirar(listaProjeteis) {}  // padrão: não atira
}

class InimigoRapido extends Inimigo {
  mover(velocidadeBase) {          // sobrescreve: 2x mais rápido
    this.y += velocidadeBase * this.multiplicadorVelocidade
              * this.direcao;
  }
}
```

```
this.inimigos.forEach((inimigo) => {
  inimigo.mover(this.velocidadeBaseInimigos);
  inimigo.tentarAtirar(this.projeteisInimigo);
});
```

UMA LISTA, TRÊS COMPORTAMENTOS



Básico

só anda em bloco



Rápido

anda 2× mais rápido



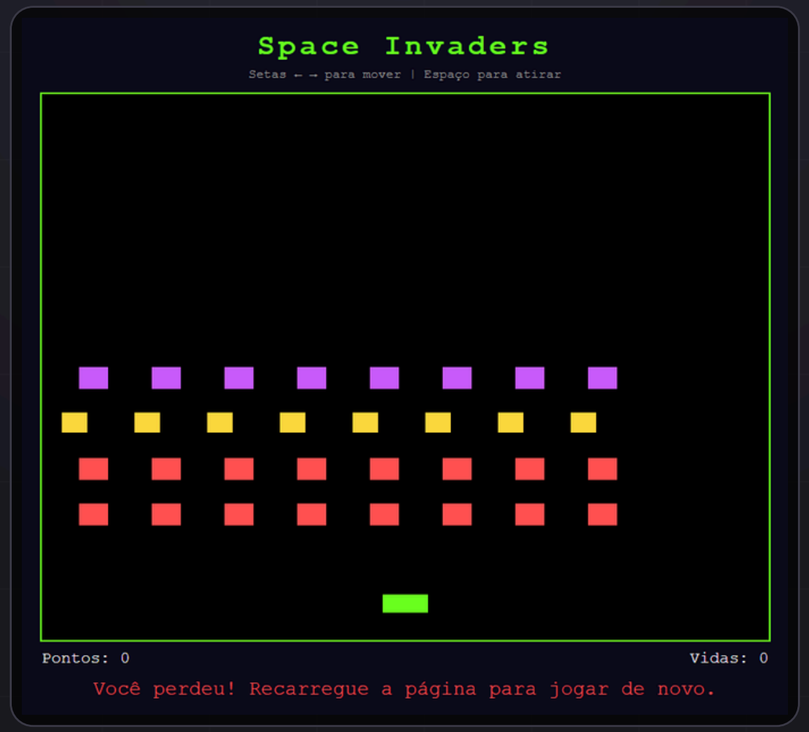
Atirador

anda e atira

O Jogo chama os mesmos métodos e não precisa saber o tipo de cada inimigo.

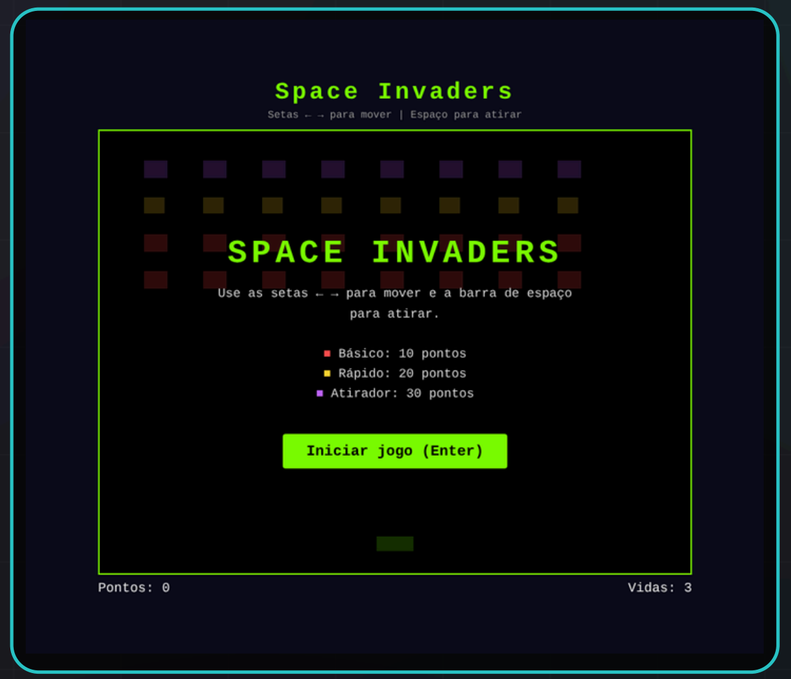
Telas de início e de fim

ANTES



O jogo começava sozinho e, no fim, pedia para recarregar a página.

DEPOIS



INÍCIO



FIM DE JOGO

Agora o jogo tem três momentos: início, jogando e fim, com botão para jogar de novo e a pontuação final.

Como funciona: um loop só

JOGO.JS

```
iniciar() {
  if (this.rodando) return;  // evita criar dois loops
  this._resetarEstado();     // zera pontos, vidas e inimigos
  this.tela.esconder();
  this.rodando = true;
  this.idFrame = requestAnimationFrame(this._loop);
}
```

Sem o if, dois cliques no botão criavam dois loops e o jogo passava a andar 2× mais rápido.

ENCAPSULAMENTO

A classe Tela guarda os elementos da página e só oferece mostrarInicio(), mostrarFim() e esconder().

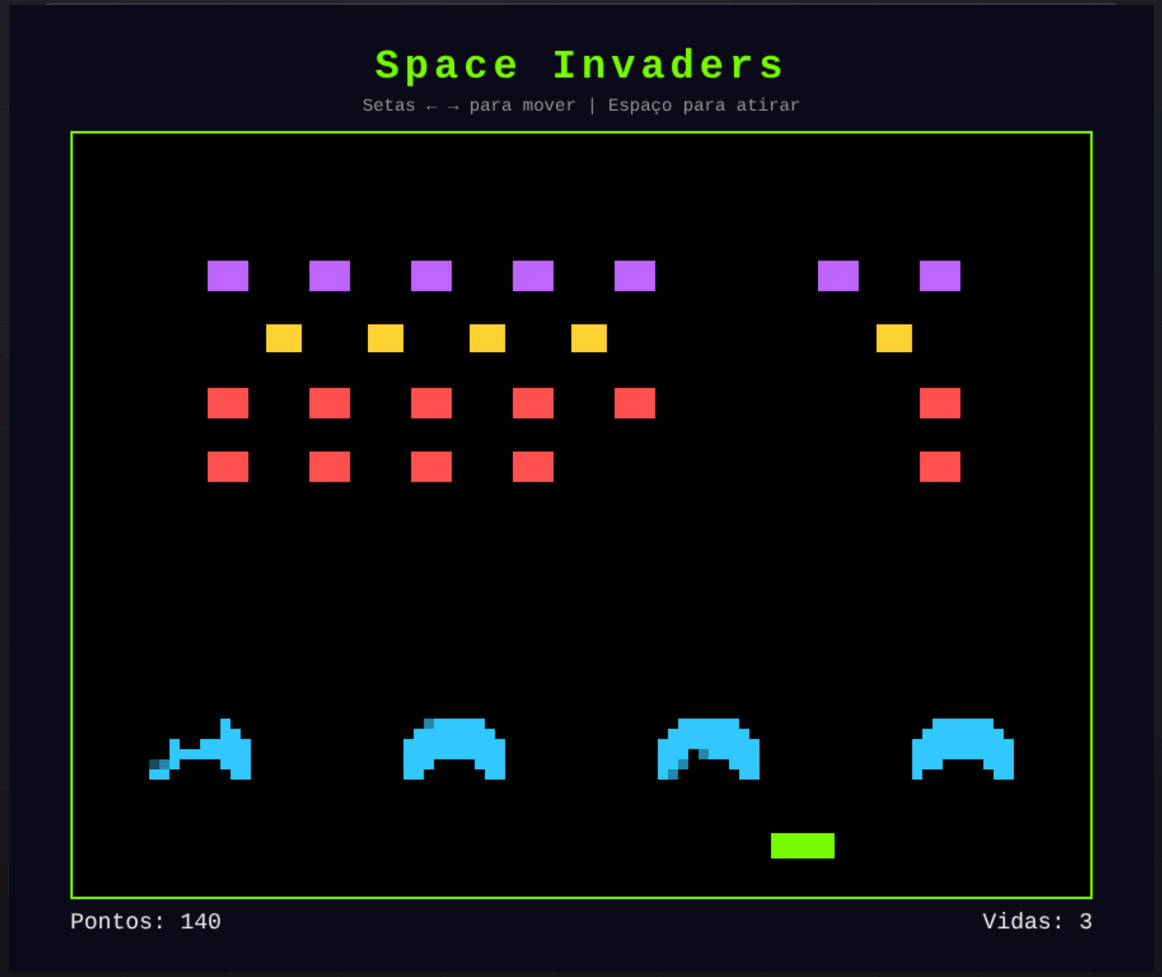
RESPONSABILIDADE ÚNICA

O Jogo cuida das regras. A Tela cuida só das mensagens.

FUNÇÃO COMO PARÂMETRO

O Jogo entrega () => this.iniciar() para a Tela chamar quando o botão é clicado.

Escudos feitos de texto



BARREIRA.CRIARESCUDO()

```
const formato = [
  '..######..',
  '.########.',
  '##########',
  '##########',
  '###....###',
  '##......##',
];  // cada # vira um bloco de 8x8 px
```

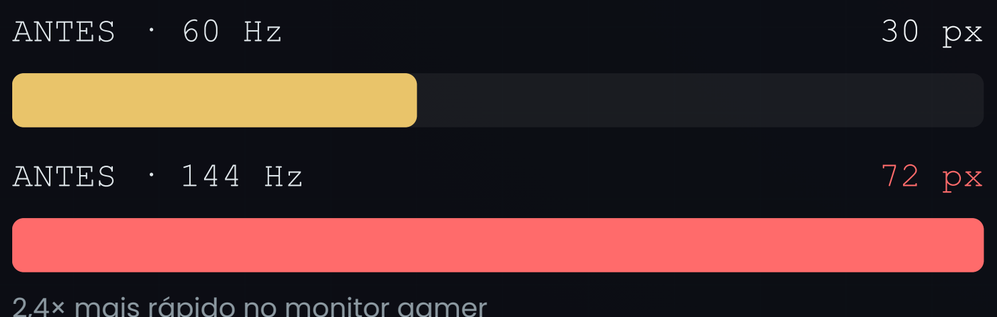
Cada bloco aguenta 3 tiros e escurece com o dano. Os inimigos destroem os blocos que tocam.

HERANÇA: BARREIRA EXTENDS ENTIDADE

MÉTODO ESTÁTICO

Velocidade em passo fixo

QUANTO O INIMIGO ANDA EM 0,5 S



APRESENTA · MARCOS

JOGO._LOOP(AGORA)

```
this.tempoAcumulado += agora - this.ultimoTempo;
this.ultimoTempo = agora;
while (this.tempoAcumulado >= this.passo) {
  this._atualizar();  // 1/60 de segundo
  this.tempoAcumulado -= this.passo;
}
```

O jogo andava um passo por quadro da tela. Agora ele mede o tempo real e roda as regras 60 vezes por segundo.

CORREÇÃO DE BUG

FIXED TIMESTEP

Pausa: três jeitos de parar



01 Teclas P ou Esc

02 Botão "Pausar (P)" no rodapé

03 Automática ao trocar de aba

JOGO.JS

```
continuar() {
  this.pausado = false;
  this.tempoAcumulado = 0;  // o tempo parado não conta
  this.ultimoTempo = performance.now();
  this.idFrame = requestAnimationFrame(this._loop);
}
```

De retângulos a pixel art



Polvo

BÁSICO · 10 PTS



Caranguejo

RÁPIDO · 20 PTS



Lula

ATIRADOR · 30 PTS

SPRITES.JS · POLVO (QUADRO 1 DE 2)

```
'....####....',
'.##########.',
'############',
'###..##..###',
'############',
'...##..##...',
'..##.##.##..',
'##........##',
```

APRESENTA · CLAYTON

INIMIGO.JS · SOBRESCREVE DESENHAR() DA ENTIDADE

```
desenhar(ctx) {
  Sprites.desenhar(ctx, this.quadros[...], ...);
}

this.quadros = Sprites.POLVO;      // InimigoBasico
this.quadros = Sprites.CARANGUEJO; // InimigoRapido
this.quadros = Sprites.LULA;       // InimigoAtirador
```

Cada desenho é uma lista de linhas de texto: # é um pixel pintado. Dois quadros se alternam e criam a "marcha" dos invasores.

Virando o jogo de lado



VERTICAL · 800×600

→



LATERAL · TELA CHEIA

11 de 13

arquivos alterados

4

direções: setas ou WASD

APRESENTA · ANDRÉ

O truque: girar desenhos de texto

ANTES · 10×6



→

```
Sprites.girar(mapa,
    'horario')
```

DEPOIS · 6×10



MOVIMENTO NA DIAGONAL

0,7071

Sem esse fator, a nave andaria 41% mais rápido na diagonal.

```
const fator = direcaoX !== 0 &&
  direcaoY !== 0 ? Math.SQRT1_2 : 1;
```

A mesma função gira o canhão e os escudos. A boca do escudo passa a ficar de frente para a nave.

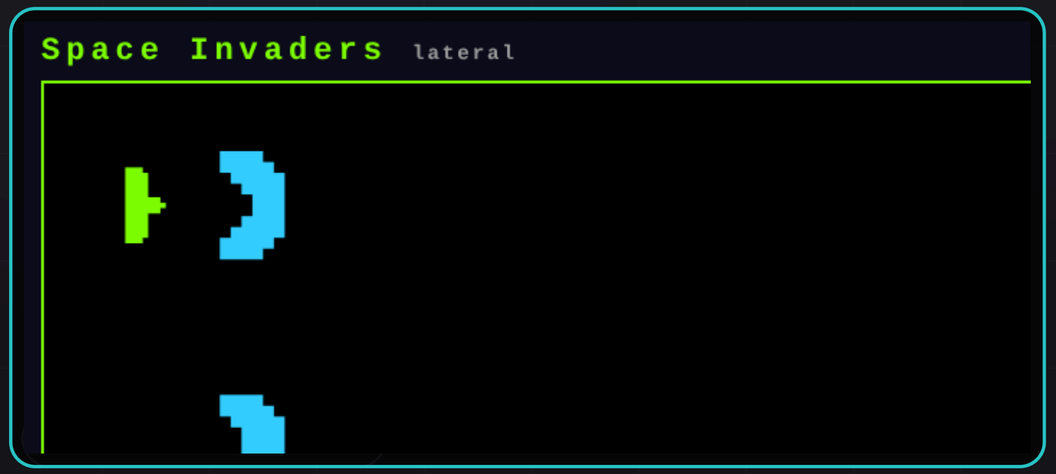
APRESENTA · ANDRÉ

Ajustes na versão lateral

TIRO DA NAVE · ANTES



DEPOIS



BUG · A NAVE ATRAVESSAVA O ESCUDO

Depois de mover a nave, o Jogo testa se ela encostou no escudo. Se encostou, tenta só o movimento vertical, depois só o horizontal; se nada der, ela fica parada. Assim a nave desliza pela borda.

O escudo agora segura todos os tiros, como no original. A nave nasce no vão entre dois escudos.

MELHORIA · SETAS IGUAIS AO WASD

```
Jogo.SETAS_PARA_WASD = {
  ArrowUp: 'KeyW',  ArrowLeft: 'KeyA',
  ArrowDown: 'KeyS', ArrowRight: 'KeyD',
};  // atributo estático: setas e WASD pelo mesmo código
```

5 hordas, uma tabela



HORDA 5 · 60 INVASORES

HORDA	COLUNAS	INVASORES	ATIRADORES
1	ARAB	32	16
2	ARABA	40	24
3	AARABA	48	32
4	AARAABA	56	40
5	AARAAB ×10	60	40

```
tipoDaLetra = { A: InimigoAtirador,
  R: InimigoRapido, B: InimigoBasico };
```

A letra escolhe a classe. A cada horda as vidas voltam para 3, mas os escudos continuam danificados.

Testamos e ajustamos

01

Fácil demais

Mais atiradores em todas as hordas. Na horda 1, de 8 para 16.

02

Horda 5 rápida demais

Bloco mais lento, avanço menor e menos tiros na última horda.

03

Impossível em tela estreita

Horda 5 de 8 para 6 colunas: de 80 para 60 invasores.

HORDA 5 · VITÓRIAS DO JOGADOR AUTOMÁTICO EM 6 PARTIDAS

JANELA	8 COLUNAS	6 COLUNAS
900×900	1/6	6/6
1024×768 (projetor)	4/6	6/6
1600×900	6/6	6/6

Jogamos, achamos o problema e pedimos a mudança. A IA não decidiu sozinha.

Bug: o jogo dependia do monitor

2,4×

mais rápido em um monitor de 144 Hz do que em um de 60 Hz.

INIMIGO EM 0,5 SEGUNDO DE JOGO

MONITOR	ANTES	DEPOIS
60 Hz	30 px	29 px
144 Hz	72 px	29 px

JOGO.JS · PASSO FIXO

```
_loop(agora) {
  this.tempoAcumulado += Math.min(agora - this.ultimoTempo, 250);
  this.ultimoTempo = agora;
  while (this.tempoAcumulado >= this.passo) {  // passo = 1000 / 60
    this._atualizar();
    this.tempoAcumulado -= this.passo;
  }
  this._desenhar();
}
```

O jogo passou a contar o tempo, e não os quadros: as regras rodam 60 vezes por segundo em qualquer monitor.

APRESENTA · MARCOS

Som e música sem nenhum arquivo

Todos os sons são criados na hora pelo próprio navegador, com a Web Audio API: tons e ruído, sem arquivos de áudio.

SOM.JS

Tiro, explosão, nave atingida, marcha dos invasores, nova horda, pausa, vitória e derrota.

MUSICA.JS

Loop de 4 acordes: Lá menor, Fá, Dó e Sol. Acelera 5 batidas por minuto a cada horda, de 100 a 120.

COMPOSIÇÃO

O Jogo tem um Som e uma Musica, e a Musica usa o Som. O Jogo só chama `som.explosao()`.

M · SOM LIGA/DESLIGA

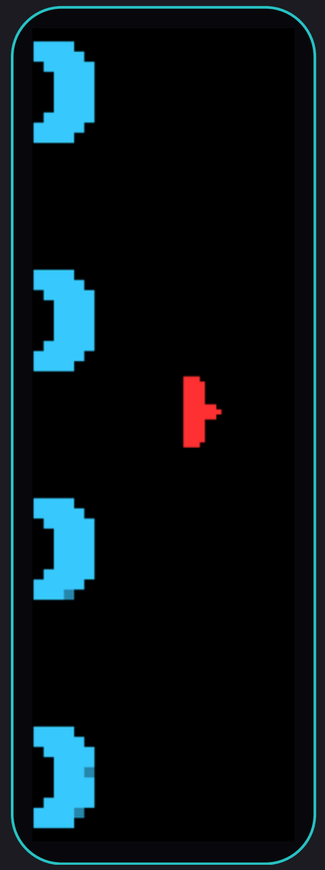
N · SÓ A MÚSICA

F · TELA CHEIA

Melhorias no dano e nos escudos



TIROS DO JOGADOR ATRAVESSAM A BARREIRA



JOGADOR ATINGIDO

MELHORIA · PISCA-PISCA DE DANO

Ao receber dano, o jogador pisca em vermelho, deixando o impacto mais visível durante a partida.

AJUSTE · POSIÇÃO DOS ESCUDOS

As barreiras foram deslocadas um pouco para a direita, ajustando o espaço entre a nave e os escudos.

MELHORIA · TIROS DA NAVE

Os tiros do jogador atravessam os escudos para alcançar a horda. As barreiras continuam fazendo parte da proteção contra os tiros inimigos.

04 · FECHAMENTO

POO no projeto

HERANÇA

Entidade → Inimigo → 3 tipos de inimigo

Inimigo.js

POLIMORFISMO

`mover()`, `tentarAtirar()` e `desenhar()` sobrescritos

InimigoRapido.js · InimigoAtirador.js

ENCAPSULAMENTO

Tela e Barreira escondem os detalhes do Jogo

Tela.js · receberDano()

ESTÁTICO

`Jogo.HORDAS` e `Barreira.criarEscudo()` pertencem à classe

Jogo.js · Barreira.js

COMPOSIÇÃO

O Jogo tem uma Tela, um Som e uma Musica

Jogo.js

RESPONSABILIDADE ÚNICA

Jogo = regras · Tela = mensagens · Som = áudio

uma classe por arquivo

APRESENTA · CLAYTON

Obrigado.









ANDRÉ SEIJI SHIROMA

MARCOS ALMEIDA DALOIA

CLAYTON SANTOS FERREIRA

ENZO MANCIO

CRÉDITOS

Claude (Anthropic) e Gemini (Google) · VS Code com Live Server · Web Audio API (MDN) · "Fix Your Timestep!", de Glenn Fiedler · inspirado no Space Invaders original (Taito, 1978). Gráficos e sons feitos por código, sem assets de terceiros.